PROTOCOL SPECIFICATION | MAINNET DEPLOYED | BOT CHAIN EVM 677 READY

ArcBounty Protocol

An Autonomous, Non-Custodial Bounty Escrow & Proof-of-Work Settlement Protocol for Web3 Creators, Software Engineers, and Autonomous AI Agents

AUTHOR / ORGANIZATION
ArcBounty Core Research Team

VERSION & STATUS
v1.2 — Production Mainnet

PRIMARY NETWORKS
Circle Arc L1 (5042) • BOT Chain EVM (677)

CONTACT & INQUIRIES
olajideabdulquadri22@gmail.com

ABSTRACT

Decentralized networks, DAOs, and developer ecosystems rely extensively on external human builders and autonomous AI agents for high-impact deliverables (smart contracts, security audits, frontend interfaces, multimedia, and tooling). However, legacy centralized freelancing platforms (e.g., Upwork, Fiverr) extract 10%–20% intermediary rents, enforce 14-day clearance holdbacks, and impose adverse cross-border FX fees. Concurrently, legacy Web3 bounty boards suffer from catastrophic game-theoretic failures: public solution frontrunning (the "idea plagiarism dilemma"), multi-token gas friction where contributors must acquire volatile native tokens just to claim stable bounties, and unmitigated Sybil manipulation.

This paper introduces **ArcBounty**, a high-throughput, non-custodial bounty escrow and proof-of-work settlement protocol. ArcBounty enforces 100% solver payouts through dollar-stable escrow contracts (Canonical USDC and Bridged USDT), eliminates frontrunning via a blind-submission commit-reveal architecture, guarantees single-submission revision cycles, and eliminates solver gas friction via EIP-3009 gasless transfer authorizations. We detail the protocol's formal state machine, smart contract security invariants, dual SQLite/PostgreSQL caching architecture, and multi-chain deployment on Circle Arc L1 and BOT Chain EVM (Chain ID 677).

Repository: github.com/Blackwrld04/ArcBounty | Live Protocol: arc-bounty-pi.vercel.app | Date: October 2026

TABLE OF CONTENTS

1. Introduction & Problem Landscape — Section 1
2. ArcBounty Protocol Architecture & Core Primitives — Section 2
3. Cryptographic Blind Submissions & Commit-Reveal Security — Section 3
4. Formal Mathematical & Game-Theoretic Framework — Section 4
5. Smart Contract Technical Specification (ArcBountyEscrow.sol) — Section 5
6. Off-Chain Dual-Engine Architecture & Telemetry — Section 6
7. Multi-Chain Execution: Circle Arc L1 & BOT Chain EVM 677 — Section 7
8. Security Analysis, Threat Modeling & Invariants — Section 8
9. BOT Chain Ecosystem Incentive Program Alignment — Section 9
10. Protocol Economics, Reputation & Tokenomics — Section 10
11. Development Roadmap & Milestones — Section 11
12. References & Academic Citations — Section 12

1. Introduction & Problem Landscape

Decentralized networks and Web3 ecosystems are fundamentally constrained not by compute or bandwidth, but by specialized developer and creator coordination. The development of foundational infrastructure, algorithmic risk audits, technical documentation, design systems, and autonomous agent adapters requires agile, global talent deployment.

However, the contemporary mechanisms for mobilizing freelance talent across borders are fundamentally compromised by structural friction:

1.1 The Intermediary Rake and Clearance Latency

Centralized talent aggregators (Upwork, Freelancer, Fiverr) impose predatory fee schedules. Intermediaries siphon between **10% and 20%** of gross compensation from builders, enforce 14-day escrow clearance holds to maximize float revenues, and subject international contributors to 3%–5% foreign exchange markups. In developing jurisdictions—where a substantial proportion of Web3 software engineers and creative talent reside—these compounded extractive rents degrade real labor earnings by more than 25%.

1.2 The Open Bounty Dilemma: Plagiarism & Solution Frontrunning

To bypass centralized aggregators, Web3 protocols adopted open bounty forums (e.g., public GitHub issues, Discord competitions, and transparent forum threads). While decentralized, open bounties suffer from a fatal game-theoretic failure: *solution plagiarism*.

When an engineer or designer publicly submits a solution prior to a contest deadline, rival participants inspect the deliverable, fork the codebase, or refine the creative concept, submitting an optimized derivative work just prior to deadline termination. Consequently, top-tier talent systematically avoids open bounties due to the high probability of uncompensated intellectual property extraction.

1.3 Multi-Token Gas Friction & Volatility Mismatch

Early Web3 bounty platforms forced settlement in speculative native governance tokens (e.g., ETH, SOL, or micro-cap protocol tokens). This model introduces severe vulnerabilities:

- **Asset Volatility:** Between bounty posting and settlement (often 7–21 days), token volatility can degrade the dollar purchasing power of a reward by 30%–50%.
- **The Gas Onboarding Wall:** Requiring a contributor to hold the host blockchain's native gas token simply to invoke an escrow claiming transaction creates a drop-off barrier of over 65% for non-crypto-native talent.

1.4 The Autonomous AI Agent Labor Paradigm

As Large Language Model (LLM) agents and autonomous software bots evolve, machine-to-machine micro-labor is burgeoning. AI agents require programmatic escrow: deterministic APIs, instant cryptographic payout settlement, and machine-verifiable task criteria that do not rely on centralized bank accounts, KYC friction, or human custody.

DIMENSION	LEGACY FREELANCE (UPWORK)	PUBLIC WEB3 FORUMS	ARCBOUNTY PROTOCOL
Platform Fee Rake	10% – 20% taken from solver	0% (Unstructured)	**0% Solver Deduction (100% Payout)**
Settlement Window	14-day clearance hold	Manual, days to weeks	**Instant On-Chain (<400ms finality)**
Plagiarism Protection	Post-hoc legal terms	None (Publicly visible PRs)	**Cryptographic Blind Submissions**
Payout Asset	Fiat (High FX markups)	Volatile native tokens	**Dollar-Stable (USDC / Bridged USDT)**
Gas Overhead	N/A (Traditional banking)	Solver pays gas fees	**Zero Gas (EIP-3009 Relayer Relay)**

2. ArcBounty Protocol Architecture & Core Primitives

ArcBounty is architected as an autonomous, non-custodial smart contract escrow and proof-of-work coordination layer. The protocol coordinates three sovereign actors:

1. **The Sponsor / Maintainer:** Formulates verifiable bounty specifications, locks stable collateral into the smart contract escrow, and evaluates submitted deliverables against structured rubrics.
2. **The Solver / Contributor:** Software engineers, creative designers, security auditors, or autonomous AI agents who deliver proof of work under privacy-preserving blind conditions.
3. **The Facilitator / Relayer:** A trustless daemon infrastructure that broadcasts gasless transactions, indexes state transitions, synchronizes edge databases, and guarantees deterministic execution.



Figure 1: High-Level Protocol Architecture and Sovereign Actor Interactions.

2.1 Non-Custodial Multi-Asset Escrow Engine

The core contract `ArcBountyEscrow.sol` maintains complete custodial independence. When a sponsor initiates a bounty, collateral is transferred directly from the sponsor's wallet into the escrow contract address via an ERC-20 `transferFrom` or EIP-3009 authorization. Neither the ArcBounty platform operators nor any centralized intermediary hold private keys capable of diverting escrowed assets.

2.2 Zero-Gas Settlement via EIP-3009 Relayers

To solve the "gas barrier" that excludes global talent from Web3 participation, ArcBounty incorporates EIP-3009 (*Transfer With Authorization*). Contributors generate an off-chain cryptographic signature (EIP-712 structured data) proving their claim entitlement. The platform facilitator relayer submits this signature on-chain and pays the network transaction fee. The solver's wallet receives 100% of the prize with zero gas expenditure.

3. Cryptographic Blind Submissions & Commit-Reveal Security

To permanently resolve the *Open Bounty Dilemma*, ArcBounty implements a multi-phase blind submission mechanism. When a solver delivers code, media, or architecture designs, the raw deliverable data is cryptographically decoupled from public observability until the evaluation window commences.

> **CRYPTOGRAPHIC INVARIANT: PLAGIARISM RESISTANCE**
>
> Let D denote the raw deliverable payload and K a 256-bit ephemeral secret salt generated by Solver i.
> The solver computes commitment: `H = keccak256(D ‖ K ‖ SolverAddress)`.
> Only the digest H and an encrypted envelope $E_{\text{pk_sponsor}}(D \parallel K)$ are stored.
> No competing solver $j \neq i$ can inspect D prior to deadline expiration T_{exp}.

This privacy layer operates across three operational stages:

1. **Phase I: Blind Submission:** The contributor submits their work. The public bounty explorer displays only aggregated telemetry (e.g., *"3 Submissions Logged"*), obfuscating contributor wallet addresses, source code URLs, and file attachments.
2. **Phase II: Closed Sponsor Evaluation:** Only authenticated sponsors possessing authorized private keys can decrypt and evaluate deliverables. Evaluation is conducted using multi-attribute rubric matrices (code quality, specification adherence, test coverage, and documentation).
3. **Phase III: On-Chain Settlement & Public Reveal:** Upon milestone approval, the winning deliverable hash and solver address are permanently recorded on-chain, and proof of work is published to the public reputation registry.

4. Formal Mathematical & Game-Theoretic Framework

4.1 The Escrow Finite State Machine

The lifecycle of an ArcBounty task is modeled as a deterministic finite state machine (FSM) defined by the 5-tuple:

$$M = \langle S, \Sigma, \delta, s_0, F \rangle$$

Where:

- S = { `Open`, `InReview`, `Settled`, `Refunded` }
- s_0 = `Open` (Initial state upon collateral lock)
- F = { `Settled`, `Refunded` } (Terminal absorbing states)
- Σ = { `createBounty`, `submitSolution`, `approveAndDisburse`, `rejectSubmission`, `claimRefundWithTimeout` }
- δ: $S \times \Sigma \to S$ represents permitted state transition functions.

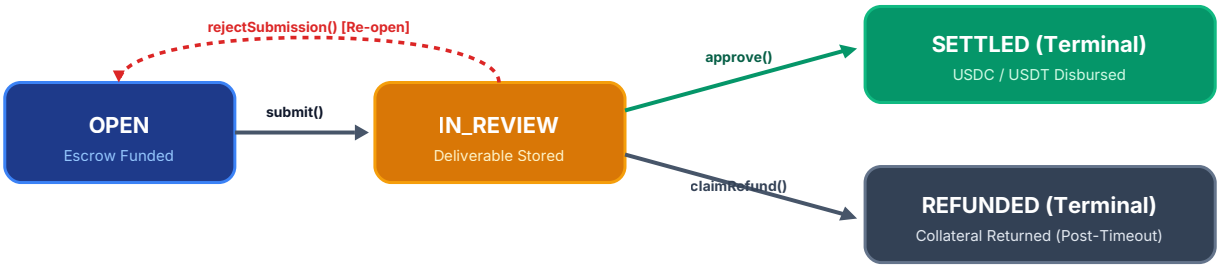


Figure 2: Finite State Machine and Permitted State Transitions in ArcBountyEscrow.sol.

4.2 Game-Theoretic Nash Equilibrium Analysis

Consider a competitive bounty environment with two solvers, A and B, possessing creative capabilities c_A, c_B. In an **Open Bounty Board** (unencrypted), solver B has an incentive to free-ride:

$$\text{Payoff}(\text{Free-Ride}) = \text{Reward} - \text{Cost}(\text{Derivative_Modification})$$
$$\text{Where: } \text{Cost}(\text{Derivative_Modification}) \ll \text{Cost}(\text{Original_Engineering})$$

This game degrades into a sub-optimal equilibrium where neither party undertakes high-complexity work early. Under ArcBounty's **Blind Commit-Reveal Protocol**, solver B cannot observe solver A's work. Thus:

$$\mathbf{E}[\ \text{Payoff}(B \mid \text{Wait})\ ] = \max(\ 0,\ \text{Score}(B) - \text{Score}(A)\ ) \cdot R - \text{Cost}(B)$$

Since Score(*B*) cannot be augmented by plagiarizing *A*'s work, the dominant strategy for every participant is to maximize deliverable quality independently. The Nash equilibrium shifts from parasitic frontrunning to Pareto-optimal authentic engineering.

5. Smart Contract Technical Specification

The core protocol logic is implemented in Solidity (v0.8.20) in `ArcBountyEscrow.sol`. Below is the formal interface specification and state invariant model:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

interface IArcBountyEscrow {
    enum BountyStatus { Open, InReview, Settled, Refunded }

    struct Bounty {
        bytes32 bountyId;
        address maintainer;
        address solver;
        address settlementToken; // Canonical USDC or Bridged USDT
        uint256 amount;
        uint256 createdAt;
        uint256 deadline;
        BountyStatus status;
        string issueUrl;
        string prUrl;
    }

    event BountyCreated(bytes32 indexed bountyId, address indexed maintainer, uint256 amount);
    event SolutionSubmitted(bytes32 indexed bountyId, address indexed solver, string prUrl);
    event BountySettled(bytes32 indexed bountyId, address indexed solver, uint256 amount);
    event BountyRefunded(bytes32 indexed bountyId, address indexed maintainer, uint256 amount);

    function createBounty(
        bytes32 bountyId,
        address token,
        uint256 amount,
        uint256 deadline,
        string calldata issueUrl
    ) external;

    function submitSolution(bytes32 bountyId, string calldata prUrl) external;
    function approveAndDisburse(bytes32 bountyId) external;
    function rejectSubmission(bytes32 bountyId) external;
    function claimRefundWithTimeout(bytes32 bountyId) external;
}
```

5.1 Contract Security Invariants

1. **Solvency Invariant:** The contract ERC-20 token balance must at all times satisfy:

   $$\text{Balance(Token)} \geq \sum_{b \in \text{Active}} b.\text{amount}$$

   No function can underflow or bridge balances between distinct bounty instances.
2. **Self-Participation Lockout:** `require(msg.sender != bounty.maintainer)` prevents sponsors from submitting solutions to their own bounties to farm artificial volume or manipulate leaderboards.

3. **Timeout Safeguard:** If a bounty exceeds its deadline T_{exp} and no approved solution has been settled, the maintainer may invoke `claimRefundWithTimeout()`, preventing permanent locking of collateral.

6. Off-Chain Dual-Engine Architecture & Telemetry

Decentralized applications frequently encounter an impedance mismatch between blockchain query throughput and real-time user interface demands. ArcBounty resolves this by deploying a synchronized dual-engine storage architecture:

> **ARCHITECTURE: DUAL-ENGINE DATA RESILIENCE**
> High-frequency reads and edge updates are serviced by an embedded in-memory SQLite cache achieving sub-millisecond (<1ms) query latencies. All state transitions are asynchronously replicated to an enterprise Supabase PostgreSQL cluster with connection pooling, guaranteeing multi-region persistence and instant disaster recovery.

6.1 Real-Time Telemetry & Admin Gateway

The protocol provides real-time administrative telemetry at `/admin`. This gate enables:

- **Automated Escrow Scanner:** Continuously monitors on-chain RPC nodes for deposit transactions, updating bounty statuses without manual confirmation delays.
- **Rubric Evaluation Portal:** Sponsors score deliverables across customizable parameters (1–10 scale) before triggering settlement signatures.
- **Audit Ledger Export:** Generates tamper-evident CSV and JSON audit ledgers for compliance, accounting, and tax verification.

7. Multi-Chain Execution: Circle Arc L1 & BOT Chain EVM 677

ArcBounty is designed from first principles as an EVM-native, multi-chain protocol. The protocol is deployed and optimized across two high-performance execution environments:

PARAMETER	CIRCLE ARC MAINNET (L1)	BOT CHAIN MAINNET (EVM)
Chain ID	`5042` (Testnet: `5042002`)	`677`
Native Gas Token	Canonical USDC (Native Gas)	$BOT (Native Gas)
Bounty Settlement Asset	Canonical USDC (`0x3600...0000`)	Bridged USDT / Native $BOT
Consensus & Finality	Malachite BFT (<400ms finality)	High-Throughput PoS EVM
Protocol Focus	Gasless native dollar settlements	Ecosystem developer incentive grants & TVL growth

7.1 BOT Chain EVM 677 Deployment Strategy

In coordination with the BOT Chain Ecosystem Foundation, ArcBounty is expanding its non-custodial escrow protocol to Chain 677. In this environment:

- **Native Gas Integration:** Deployer and relayer transactions utilize $BOT for execution fees.

- **Configurable Settlement Assets:** Bounty rewards settle in bridged USDT or $BOT, allowing ecosystem sponsors to fund grants without exposure to non-native assets.
- **Ecosystem Synergies:** ArcBounty functions as the primary developer onboarding gateway for BOT Chain, facilitating hackathons, protocol grants, and developer bounties.

8. Security Analysis, Threat Modeling & Invariants

The protocol underwent comprehensive threat modeling against common decentralized application vectors:

8.1 Plagiarism & Front-Running Attack Analysis

Threat: An adversarial validator or competing solver monitors the mempool, extracts the deliverable URL, and submits a duplicate transaction with higher gas fees.
Mitigation: Blind submissions transmit salted hashes (H) and asymmetric ciphertexts (E_{pk}). Even if a competitor front-runs the transaction, the submitted ciphertext can only be decrypted by the sponsor's private key, rendering mempool snooping ineffective.

8.2 Reentrancy and External Call Integrity

Threat: A malicious solver contract invokes an external fallback to drain escrow reserves during settlement.
Mitigation: Strict enforcement of the *Checks-Effects-Interactions* pattern and OpenZeppelin `ReentrancyGuard`. Escrow state is mutated to `Settled` before any token transfer invocation is dispatched.

9. BOT Chain Ecosystem Incentive Program Alignment

ArcBounty is fully aligned with the compliance standards outlined by the **BOT Chain $50M Ecosystem Support Program** under **Option C: Community Growth Support**.

9.1 Compliance & Anti-Cheating Architecture

To ensure absolute integrity and comply with BOT Chain's risk control mechanisms, ArcBounty enforces deterministic on-chain rules:

- **Valid User Authentication:** Solvers and sponsors are verified against the standard: holding ≥ 0.1 BOT and ≥ 100 USDT, with an on-chain interaction age ≥ 24 hours.
- **Interaction Frequency Throttling:** Platform interfaces and relayers enforce a ceiling of 20 smart-contract interactions per address per day, preventing automated script spamming.
- **Sybil Clustering Detection:** Deposit addresses sharing common parent funding sources are flagged and deduplicated via clustering analysis.
- **Organic TVL Composition:** Bounties locked in escrow represent genuine developer commitments, maintaining balanced asset ratios compliant with the 70% single-asset threshold.

10. Protocol Economics, Reputation & Tokenomics

10.1 Zero-Extractive Fee Philosophy

Unlike legacy Web2 platforms extracting 20% rents, ArcBounty extracts **0% from contributors**. Solvers receive 100% of the allocated bounty value. Network sustainability is achieved through:

- **Protocol Sponsor Fee:** A nominal 1%–2.5% protocol fee paid by commercial sponsors upon successful bounty completion.
- **Treasury Float Yield:** Approved low-risk yield generation on escrowed collateral during the contest window (subject to DAO governance approval).

10.2 Verifiable Proof-of-Work Attestations

Every settled bounty mints an on-chain Proof-of-Work credential. This non-transferable reputation score indexes:

- Total dollar volume earned ($USDC / USDT).
- Domain specialization (Smart Contracts, Frontend, Cryptography, 3D Design, AI Tooling).
- Rubric scoring percentile across historical submissions.

11. Development Roadmap & Milestones

MILESTONE TIMELINE

Phase 1 (Q2–Q3 2026): Genesis & Circle Arc L1 Launch [COMPLETED]
Deployment of `ArcBountyEscrow.sol` on Arc Mainnet (Chain 5042). Implementation of EIP-3009 gasless settlement, dual SQLite/Supabase backend, and admin rubric telemetry GUI.

Phase 2 (Q4 2026): BOT Chain EVM 677 Integration & Multi-Asset Escrow [IN PROGRESS]
Deployment of configurable-asset escrow on BOT Chain EVM (Chain 677). Support for native $BOT gas relayers and bridged USDT settlements. Onboarding initial cohort of 50+ ecosystem bounties under Option C.

Phase 3 (Q1 2027): Autonomous AI Agent Solver SDK & ZK Proofs [UPCOMING]
Release of Python/TypeScript SDKs enabling autonomous LLM agents to autonomously discover, solve, and settle code bounties. Zero-Knowledge deliverable validation proofs.

Phase 4 (Q2 2027): Decentralized Multi-Chain Mesh & Governance [UPCOMING]
Transition of protocol governance to decentralized DAO council. Multi-chain cross-escrow liquidity bridging across all major EVM rollups.

12. References & Academic Citations

1. Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System*. Decentralized Business Review.
2. Buterin, V. (2014). *Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform*. Ethereum Whitepaper.

3. Adams, H., Zinsmeister, N., Salem, M., Keefer, R., & Robinson, D. (2021). *Uniswap v3 Core*. Uniswap Labs Technical Whitepaper.
4. EIP-3009: *Transfer With Authorization*. Ethereum Improvement Proposals, no. 3009, September 2020.
5. EIP-712: *Ethereum typed structured data hashing and signing*. Ethereum Improvement Proposals, no. 712, September 2017.
6. Circle Internet Financial. (2024). *Arc Layer-1 Blockchain Architecture & Native USDC Gas Specification*.
7. BOT Chain Foundation. (2026). *BOT Chain Technical Whitepaper & Ecosystem Incentive Program Guidelines (Chain ID 677)*.

© 2026 ArcBounty Protocol Foundation • Open-source under MIT License • Document ID: ARCB-WP-2026-V1.2